

TP 1 Analyse Numérique

La version interactive de ce TP se trouve ici :

[ce lien](#)

Cette version interactive vous permettra de copier-coller le python plus facilement !

Point fixe, Dichotomie, Newton, Interpolation polynomiale, Calcul approché d'intégrales.

Consignes

Intelligence artificielle interdite dans votre rendu final.

- Soupçons -> Division de la note par 2
- Certitude -> 0/20
- Vous pouvez vous en servir pour demander des exemples simples ou poser des questions.

Vous rendrez un rapport de TP avec

- les réponses aux questions,
- les parties de code que vous avez modifiée ou écrite en les expliquant,
- les graphes que vous avez tracés.

Vous reprendrez la structure de ce sujet avec les titres et les sous-titres.

Vous pouvez travailler seul ou par binôme.

Vous rendrez votre rapport en format **pdf**, avec les noms des membres du binôme dans le nom du fichier avant le 4 Mai 23h59 en déposant votre fichier sur ce lien :

[Dépôt TP1](#)

Certaines questions sont étoilées, passez-les puis revenez-y lorsque vous aurez terminé toutes les questions non-étoilées. Il n'est pas nécessaire de répondre à toutes les questions pour obtenir une bonne note.

Posez des questions si vous êtes coincés. Pendant le TP ou par email.

Petits rappels de Python.

Pour compiler votre code, vous pouvez utiliser au choix

- Spyder
- N'importe quel éditeur de texte, puis compiler depuis le terminal.

Des bibliothèques utiles.

Tracer des graphes

Pour tracer des fonctions, on peut utiliser numpy et matplotlib.

Il vous suffira d'appeler la fonction suivante pour tracer un graphe:

```
import numpy as np
import matplotlib.pyplot as plt

def plot_function(f, a, b, n=100):
    x_plot = np.linspace(a, b, n)
    # tableau de n points entre a et b
    y_plot = f(x_plot)
    plt.plot(x_plot, y_plot)
```

En fonction de la façon dont vous compilez python, il vous faudra peut-être ajouter la ligne suivante après l'appel des fonctions :

```
plt.show()
```

Questions :

1. Représentez le graphe de la fonction sinus (en appelant la fonction avec `f = np.sin`) sur l'intervalle $[0, 2\pi]$
2. Que se passe-t-il si on change la valeur de `n` ? Testez avec `n=10` et `n=1000` et notez vos observations. Faudra-t-il une valeur de `n` plus ou moins élevée si on veut afficher un intervalle plus grand ?

Vous pouvez aussi créer des fonctions à la main puis les tracer avec le code précédent.

```
def f(x):
    return x**2 - 2

plot_function(f, -1, 1)
```

Questions :

1. Modifiez la fonction `f` pour tracer une parabole plus large sans changer l'abscisse du sommet. Afficher les deux paraboles sur le même graphe en appelant deux fois la fonction `plot_function`
2. ** Ecrivez une fonction qui prend en paramètre une fonction f , un point d'abscisse x et qui trace la tangente à la courbe de f en x .

Si vous avez besoin de sauvegarder un graphe, vous pouvez le faire avec la fonction `savefig`.

```
plt.savefig('fonctions_trigo.png')
```

Chaînes de caractères et f-strings

Pour afficher des variables ou un message dans la console, on utilise usuellement `print`, comme par exemple :

```
print("Bonjour")
```

On peut mélanger du texte et des variables avec des f-strings, il suffit d'ajouter la lettre `f` devant les guillemets :

```
a = 5
b = 3
print(f"La somme de {a} et {b} est {a + b}")
```

On peut aussi modifier le nombre de décimales affiché :

```
print(f"Pi avec 2 décimales : {np.pi:.2f}")
print(f"Pi avec 4 décimales : {np.pi:.4f}")
```

ou manipuler des expressions sans avoir à les stocker dans des variables

```
a = 4.5
print(f"Le carré de {a} est {a**2}")
print(f"La racine carrée de {a} est {a**0.5:.2f}")
```

Questions :

1. Ecrivez le code pour afficher la valeur de e^x en $x = 1$ avec 5 décimales. (vous pourrez utiliser la fonction `np.exp` de numpy)
2. Créez une fonction avec une boucle qui affiche la table de multiplication du nombre 7 en utilisant les f-strings.
3. Enveloppez cette boucle dans une autre boucle pour afficher toutes les tables de multiplication de 1 à 9.

Mesurer un temps d'exécution

Au cours de ce TP, il vous sera demandé plusieurs fois de comparer des performances d'algorithmes, pour cela, vous pourrez utiliser la fonction `time.time()`. Par exemple :

```
import time

start_time = time.time()

### Votre fonction ###

end_time = time.time()

print(f"Temps d'évaluation : {end_time - start_time} secondes")
```

1. Mesurer le temps d'exécution de votre boucle précédente qui affiche les tables de multiplication, vous pouvez augmenter le temps en affichant plus de tables de multiplications.

Des erreurs d'arrondi

Exécutez le code suivant et observez le résultat. Qu'en pensez-vous ?

```
import math

print(f"0.1 + 0.2 = {0.1 + 0.2}")
print(f"0.1 + 0.2 == 0.3 ? {0.1 + 0.2 == 0.3}")

print(f"(racine de 2)^2 = {math.sqrt(2)**2}")
```

Questions :

1. Pourquoi $0.1 + 0.2$ ne donne-t-il pas exactement 0.3 ? Expliquez en 2 phrases, *vous pouvez faire des recherches*.
2. Créez une boucle qui additionne 0.1 un million de fois dans une variable `s`, affichez le résultat théorique et la valeur obtenue.

D'autres "erreurs" de calculs :

```
# Avec des grands nombres :
grand_nombre = 1e16
print(f"1e16 + 1 = {grand_nombre + 1}")
print(f"1e16 + 1 == 1e16 ? {grand_nombre + 1 == grand_nombre}")

# Avec des nombres très proches
a = 1.0000000000000001
b = 1.0000000000000002
print(f"a = {a}")
print(f"b = {b}")
print(f"a == b ? {a == b}")
```

3. Les trois méthodes du chapitre suivant sont **itératives** : elles utilisent le résultat d'un calcul comme point de départ pour le calcul suivant. A quoi faudra-t-il faire attention ?

Méthodes itératives de résolution d'équations

Lors des premiers CM, nous avons vu 3 méthodes pour résoudre des équations de la forme $f(x) = 0$.

Méthode du point fixe

```
import numpy as np
import matplotlib.pyplot as plt
```

```
def iterative_process(x0, n):
    x = np.zeros(n) # Créer un tableau de n cases rempli de 0

    x[0] = x0

    for i in range(1, n):
        x[i] = np.cos(x[i-1]) # Ligne principale

    return x

x0 = 0.5
n = 50

x = iterative_process(x0, n)

plt.plot(x)
```

Questions :

1. Lisez, exécutez puis expliquez le code suivant.
2. Examinez la ligne principale, pour quelle équation ce code essaie-t-il de trouver une solution ?
3. Tracer les graphes des fonctions $y = x$ et $y = \cos x$ et pointez sur le graphe la solution de l'équation.
4. Testez différentes valeurs initiales `x0`. Observez-vous toujours la convergence ? Essayez des valeurs extrêmes. Vous pouvez toutes les tracer sur le même graphe.
5. * Créez une fonction qui calcule le nombre d'itérations nécessaires pour atteindre une précision ε que vous passerez en paramètre.

Le code suivant compare deux suites régies par la même relation de récurrence, mais avec une légère différence dans la valeur initiale.

```
def iterative_process(x0, n):
    x = np.zeros(n)
    x_approx = np.zeros(n)

    x[0] = x0
    x_approx[0] = x0 + 1e-10

    for i in range(1, n):
        x[i] = 30*np.cos(x_exact[i-1])
        x_approx[i] = 30*np.cos(x_approx[i-1])

    return x, x_approx

x0 = 0.2
n = 50

x_exact, x_approx = iterative_process(x0, n)
```

```
differences = np.abs(x_exact - x_approx)

plt.semilogy(differences, label="Erreur d'approximation")
```

Questions :

1. Quelle équation cet algorithme permet-il de résoudre ?
 2. Pourquoi la suite diverge-t-elle alors que l'algorithme précédent convergeait, qu'elle que soit la valeur initiale ? Rappelez l'hypothèse du théorème du cours utile dans ce contexte.
 3. * Analysez la stabilité de la méthode en fonction de la valeur de la perturbation.
-

Considérons la suite définie par :

$$\begin{cases} x_0 = 1 \\ x_{n+1} = \frac{1}{2} \left(x_n + \frac{2}{x_n} \right) \end{cases}$$

Questions :

1. Vers quoi devrait converger cette suite ?
2. La fonction f respecte-t-elle l'hypothèse du théorème ?
3. En vous inspirant du code ci-dessus, écrivez un programme qui affiche l'évolution de la suite.
4. * La vitesse de convergence dépend-elle fortement de la valeur initiale ? Vous pourrez illustrer votre réponse avec un graphique.

Dichotomie

Une autre méthode pour résoudre $f(x) = 0$ est de se fixer un intervalle initial $[a,b]$ et de découper cet intervalle en deux, en considérant uniquement la moitié qui contient la solution, puis de découper cette moitié en deux, etc ...

Voici le code que nous avons élaboré en cours pour résoudre l'équation $\cos(x) = x$ par dichotomie.

```
import math

def dichotomie(a, b, epsilon=1e-6):

    while (b - a) / 2 > epsilon:
        c = (a + b) / 2

        if c > math.cos(c):
            b = c
        else:
            a = c

    return (a + b) / 2
```

```
print(dichotomie(0, 1))
```

Questions :

1. Modifiez la fonction pour qu'elle affiche le nombre d'itérations nécessaires.
2. Comparez la vitesse de convergence avec la méthode du point fixe. *Vous pourrez par exemple tracer sur le même graphe les deux suites.*
3. Créez une fonction qui trace l'évolution de l'intervalle $[a,b]$ à chaque itération.
4. Utiliser la méthode de dichotomie pour trouver une valeur approchée de la solution de l'équation $xe^x = 1$ à 10^{-6} près.

Méthode de Newton

La troisième méthode que nous avons vu en cours pour résoudre $f(x) = 0$ est la méthode de Newton.

- On part d'une valeur initiale x_0
- On calcule la tangente à la fonction f en x_0
- On regarde l'abscisse du point d'intersection de la tangente avec l'axe des abscisses
- On recommence avec cette abscisse.

Voici le code que nous avons élaboré en cours pour résoudre l'équation $\cos(x) = x$ par la méthode de Newton.

```
import math
x0 = 0.5

def f(x):
    return x - math.cos(x)

def df(x): # dérivée de f
    return 1 + math.sin(x)

def newton(x0, epsilon=1e-6):
    x = x0

    while abs(f(x)) > epsilon:
        x = x - f(x) / df(x)

    return x

print(newton(x0))
```

Questions :

1. Modifiez le code pour afficher le nombre d'itérations nécessaires pour atteindre la précision 10^{-6} .

2. Testez différentes valeurs initiales. La méthode converge-t-elle toujours ?
3. * Que se passe-t-il si $f'(x) = 0$?
4. * Implémentez la méthode de Newton pour trouver une valeur approchée de la solution de l'équation $xe^x = 1$ à 10^{-6} près.
5. * La variable `epsilon` correspond-elle à la précision de la valeur approchée ? Y a-t-il une différence sur cette précision entre la méthode de Newton et les deux autres méthodes précédentes ?

Applications

Voici deux problèmes :

- Calculer $\sqrt[3]{231567}$.
 - En reliant le nombre π à une équation sur la fonction sinus, déterminer une valeur approchée de π à 10^{-6} près.
1. * Implémentez les trois méthodes et comparez leur vitesse de convergence. Vous pourrez par exemple
 - Un même nombre d'étapes et comparer la précision.
 - Mesurer le temps pour atteindre une précision fixée.
 2. Créez un graphique montrant l'évolution de l'erreur pour chaque méthode.
 3. Analysez la stabilité numérique de chaque méthode.

Interpolation Polynomiale

Je vous rappelle qu'un des buts principaux de notre cours est de pouvoir approcher une fonction compliquée par une fonction facile à calculer.

Aussi, les polynômes sont des fonctions faciles à calculer.

Manipulation de polynômes

On peut considérer qu'un polynôme est la liste de ses coefficients. Par exemple :

```
coefficients = [2, -6, 2, -1]
# Correspond au polynôme  $2x^3 - 6x^2 + 2x - 1$ 
```

Les fonctions suivantes prennent un polynôme P sous la forme d'une liste de coefficients et une abscisse x et renvoient la valeur de $P(x)$.

On peut évaluer un polynôme de manière naïve avec la fonction suivante :

```
def evaluer_polynome(coefficients, x):
    resultat = 0
    degre = len(coefficients) - 1
    for i, coeff in enumerate(coefficients):
```



```
resultat += coeff * (x ** (degre - i))
return resultat
```

Nous avons aussi mentionné l'algorithme de Horner pour évaluer un polynôme de manière plus efficace :

$$P(x) = \left(((a_n \times x + a_{n-1}) \times x + a_{n-2}) \times x + \dots + a_1 \right) \times x + a_0$$

Voici une fonction qui implémente ce calcul :

```
def evaluer_polynome_horner(coefficients, x):
    resultat = 0
    for coeff in coefficients:
        resultat = resultat * x + coeff
    return resultat
```

Questions :

1. Générer un polynôme de degré 1000 et un point x et mesurer le temps d'évaluation avec les deux méthodes. Vous pourrez par exemple utiliser cette fonction qui génère un tableau de 10 nombres entiers entre -5 et 5.

```
coeff = np.random.randint(-5, 5, 10)
```

2. Le calcul de l'image d'un polynôme en 0 est-il différent du calcul de l'image d'un polynôme en 1 ?
3. * Tracer le graphe de polynômes aléatoires de degré différents pour illustrer le "nombre de bosses" d'un polynôme.

Tronquer le Développement de Taylor

Pour approcher une fonction f par un polynôme, une première idée peut être de tronquer son développement de Taylor en un point x_0 :

La fonction e^{-x} sur l'intervalle $[0, 10]$ approchée par son DL en 0.

$$e^{-x} = \sum_{k \geq 0} \frac{(-x)^k}{k!}$$

Le code suivant calcule les premiers termes du développement de Taylor :

```
def taylor(x, n):
    return sum((-x)**k / np.math.factorial(k) for k in range(n))
```

Questions :

1. Pour $n = 10$, l'approximation obtenue est-elle satisfaisante ?

2. Essayez plusieurs valeurs de n pour obtenir une approximation suffisante de la fonction.
3. Mesurez le temps d'exécution du code pour ces valeurs de n .
4. En écrivant une relation de récurrence entre les termes de la somme, calculez la somme de manière plus efficace.

Interpolation de Lagrange

Ecrivez une fonction qui prend en paramètres n points et une abscisse x et renvoie la liste des n polynômes de Lagrange ℓ_i associés:

```
def lagrange(points, x):
    n = len(points)
    polys = np.zeros(n)
    for i in range(n):
        num = 1
        denom = 1
        for j in range(n):
            if i != j:
                num *= x - points[j]
                denom *= points[i] - points[j]
        polys[i] = num / denom
    return polys
```

Questions :

1. Que représente le tableau `polys` que renvoie la fonction et comment l'utiliser ?
2. Ecrivez une fonction qui prend en paramètre une fonction f et une liste de points et une abscisse x et qui renvoie la valeur en x du polynôme de Lagrange qui coïncide avec f sur chaque abscisse du tableau points.
3. Analysez l'erreur d'interpolation pour différentes fonctions que vous choisirez.
4. Comparez les résultats avec la méthode précédente qui tronque le développement de Taylor.
5. * Tracer le polynôme de Lagrange qui coïncide avec la fonction suivante aux points d'abscisses $-1, 0, 1$. Tracer la fonction sur le même graphe, ajoutez des points pour essayer de faire coller le polynôme à la fonction.

$$f(x) = \frac{1}{1+x^2}, \quad -5 \leq x \leq 5$$

Intégration numérique

Méthode des rectangles

Un des autres buts de ce cours est de pouvoir obtenir des valeurs approchées pour des intégrales compliquées.

Le code suivant implémente la méthode des rectangles à gauche pour calculer l'intégrale d'une fonction f entre a et b en l'approchant par n rectangles:

```
def integrale(f, a, b, n):
    h = (b - a) / n
    extremites = np.linspace(a, b, n+1)
    somme = 0
    for i in range(n):
        AireRectangle = f(extremites[i]) * h
        somme += AireRectangle
    return somme

print(integrale(lambda x: x**2, 0, 1, 1000))
```

Questions :

1. Choisissez une fonction dont vous pouvez calculer l'intégrale à la main et comparez le résultat théorique avec cette méthode.
2. * Illustrez sur un graphe la convergence de la méthode lorsque le nombre de rectangles augmente.
3. * Créez une fonction qui adapte automatiquement le nombre de subdivisions pour atteindre une précision donnée.

Méthode des trapèzes

1. Implémentez la méthode des trapèzes et comparez-la avec la méthode des rectangles. Vous pourrez par exemple mesurer le temps d'exécution pour atteindre une précision donnée.
2. * Modifiez le code pour utiliser des subdivisions non uniformes.
3. Analysez l'erreur en fonction du nombre de subdivisions.

Approcher par un polynôme.

Nous avons également vu qu'on peut estimer l'intégrale d'une fonction en calculant à la place l'intégrale d'un polynôme qui approche cette fonction. Cette stratégie avait conduit à la quadrature de Newton-Cotes :

```
def newton_cotes(f, a, b):
    return (b - a) / 6 * (f(a) + 4*f((a+b)/2) + f(b))
```

Questions :

1. Comparez les résultats avec les méthodes précédentes.
2. Nous avons aussi utilisé cette formule en séparant l'intervalle d'intégration en plusieurs sous-intervalles et en appliquant cette formule sur chaque sous-intervalle. Ecrivez une fonction pour calculer l'intégrale d'une fonction avec en paramètre le nombre de sous-intervalle sur lesquels appliquer cette formule.
3. Comparer les performances de cette dernière méthode avec les précédentes.
4. A partir de la liste des coefficients d'un polynôme, déterminer son intégrale exacte sur un intervalle $[a, b]$. Utilisez cette expression pour obtenir une autre façon d'obtenir une valeur

approchée d'intégrale.

5. * En utilisant les fonctions précédentes, calculer des intégrales en approchant les fonctions par des polynômes de degré supérieur.
6. * Analysez l'erreur en fonction du degré du polynôme.
7. Créez une fonction qui choisit automatiquement le degré optimal du polynôme.

Pour aller plus loin : les polynômes de Hermite.

Les polynômes de Hermite permettent d'obtenir un polynôme qui colle mieux à une fonction puisqu'on peut également approcher sa dérivée:

Si ℓ_i est le polynôme de Lagrange associé au point x_i , on définit les polynômes

$$H_i(x) = (1 - 2(x - x_i)\ell'_i(x_i))\ell_i^2(x)$$

et

$$G_i(x) = (x - x_i)\ell_i^2(x)$$

$$\begin{cases} H_i(x_j) = \delta_{ij} \\ H'_i(x_j) = 0 \end{cases}, \quad \begin{cases} G_i(x_j) = 0 \\ G'_i(x_j) = \delta_{ij} \end{cases}$$

Questions :

1. Implémentez les polynômes de Hermite et testez-les sur différentes fonctions.
2. Comparez les résultats avec l'interpolation de Lagrange.
3. Analysez l'erreur d'interpolation pour différentes fonctions.
4. Créez une fonction qui combine interpolation et dérivées pour obtenir une meilleure approximation.
5. Ecrivez un programme qui calcule l'intégrale d'une fonction f en se basant sur les polynômes de Hermite.